

Arrays, Multidimensional Arrays & Pointers

Q. Define Arrays. Explain one-dimensional arrays, multi-dimensional arrays, and the concept of an array of pointers with syntax and examples.

📌 Definition to Remember

An **Array** is a linear data structure that stores a **fixed-size, sequential collection of elements of the same data type** in **contiguous memory locations**. Elements are accessed randomly using an **index** in $O(1)$ time.

1. One-Dimensional (1D) Array

- Simplest form — a single list of elements.
- Uses **one index** to access elements.
- **Syntax:** `data_type array_name[size];`
- **Memory:** Elements stored in consecutive memory locations.

```
int arr[3] = {10, 20, 30};  
// Access: arr[0] = 10, arr[1] = 20, arr[2] = 30
```

2. Multi-Dimensional Arrays

- An array of arrays. Most common: **2D Array** (matrix — rows and columns).
- Uses **two indices**: `array[row][column]`.
- **Syntax:** `data_type array_name[rows][columns];`
- Memory storage: **Row-Major Order** (C stores row by row) or **Column-Major Order**.

```
int matrix[2][2] = {  
    {1, 2},  
    {3, 4}  
};  
// Access: matrix[1][1] = 4
```

```
Memory (Row-Major):  1 | 2 | 3 | 4  
Index:               [0][0][0][1][1][0][1][1]
```

3. Array of Pointers

- An array where each element is a **pointer** (holds a memory address).
- **Syntax:** `data_type *array_name[size];`
- Most useful for storing **arrays of strings** (`char*`) — avoids wasting memory with fixed-size 2D char arrays.

```
char *names[3] = {"Alice", "Bob", "Charlie"};
// names[0] → "Alice", names[1] → "Bob"
```

4. Key Properties of Arrays

Property	Detail
Contiguous Memory	All elements stored side by side
Random Access	O(1) access using index
Fixed Size	Cannot grow or shrink dynamically
Homogeneous	Stores only one data type

★ Must-Write Points (for 10 marks)

1. Array = fixed-size, contiguous, homogeneous, index-based data structure.
2. 1D Array: single list, one index, `int arr[5]` .
3. 2D Array: matrix (rows × columns), two indices, stored in Row-Major Order in C.
4. Array of Pointers: each element is a pointer — used for dynamic strings.
5. Array access is O(1) because of contiguous memory and direct index calculation.
6. Arrays have **fixed size** — cannot resize at runtime (unlike dynamic structures).
7. 2D arrays are used in matrix operations, graphics, and machine learning.

⚡ Quick Recall

Array → Contiguous + Fixed + Homogeneous → 1D (single index) → 2D (rows×cols, Row-Major) → Array of Pointers (*arr[n], strings) → O(1) access



Algorithmic Implementations: 2D Arrays & Sparse Matrix (C Code)

1. Matrix Multiplication Algorithm & C Code

```
#include <stdio.h>

#define R1 2
#define C1 3
#define R2 3
#define C2 2

void multiplyMatrices(int A[R1][C1], int B[R2][C2], int C[R1][C2]) {
    for (int i = 0; i < R1; i++) {
        for (int j = 0; j < C2; j++) {
            C[i][j] = 0;
            for (int k = 0; k < C1; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

int main() {
    int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int B[3][2] = {{7, 8}, {9, 1}, {2, 3}};
    int C[2][2];

    multiplyMatrices(A, B, C);

    printf("--- Resultant Product Matrix (2x2) ---\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d\t", C[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

2. Sparse Matrix Triplet Representation (C Code)

```
#include <stdio.h>

struct Element {
    int row;
    int col;
    int val;
};

struct Sparse {
    int m;
    int n;
    int num;
    struct Element ele[20];
};

void createSparse(struct Sparse *s, int matrix[4][4]) {
    s->m = 4;
    s->n = 4;
    s->num = 0;

    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            if (matrix[i][j] != 0) {
                s->ele[s->num].row = i;
                s->ele[s->num].col = j;
                s->ele[s->num].val = matrix[i][j];
                s->num++;
            }
        }
    }
}

void displaySparse(struct Sparse s) {
    printf("Row\tCol\tVal\n");
    printf("-----\n");
    for (int i = 0; i < s.num; i++) {
        printf("%d\t%d\t%d\n", s.ele[i].row, s.ele[i].col, s.ele[i].val);
    }
}

int main() {
    int matrix[4][4] = {
        {0, 0, 3, 0},
        {0, 0, 0, 0},
        {0, 5, 0, 0},
        {0, 0, 0, 9}
    };

    struct Sparse s;
    createSparse(&s, matrix);

    printf("--- Sparse Matrix Triplet Representation ---\n");
    displaySparse(s);
}
```

```
return 0;  
}
```